

Cardiovascular Disease Risk Prediction Using Machine Learning

10-Year Coronary Heart Disease Risk Estimation via Calibrated Machine Learning on the Framingham Cohort

Author	Akshay Bawaliwale
Version	1.0
Date	May 2026
Dataset	Framingham Heart Study · 3,390 patients
Best Model	Logistic Regression (Platt-calibrated)
ROC-AUC	0.714 · Sensitivity 72.6 % · Specificity 58.0 %
Repository	github.com/AkshayAI007/Cardiovascular-disease-risk-prediction-using-Machine-learning

TABLE OF CONTENTS

- 1. Project Overview
- 2. Dataset
 - 2.1 Feature Inventory
 - 2.2 Missing Values
- 3. System Architecture
- 4. Methodology
 - 4.1 Exploratory Data Analysis
 - 4.2 Data Preprocessing Pipeline
 - 4.3 Feature Engineering
 - 4.4 Class Imbalance Handling
 - 4.5 Model Training & Hyperparameter Tuning
 - 4.6 Stacking Ensemble
 - 4.7 Decision Threshold Optimisation
 - 4.8 Probability Calibration
- 5. Model Evaluation
- 6. Inference Pipeline
- 7. API Reference
- 8. Web Interface
- 9. Deployment
- 10. Project Structure
- 11. Reproducibility
- 12. Limitations & Future Work
- 13. References

TECHNICAL DOCUMENTATION · V1.0 · MAY 2026

Table of Contents

- 1. Project Overview
- 2. Dataset
- 3. System Architecture
- 4. Methodology
 - 4.1 Exploratory Data Analysis
 - 4.2 Data Preprocessing Pipeline
 - 4.3 Feature Engineering
 - 4.4 Class Imbalance Handling
 - 4.5 Model Training & Hyperparameter Tuning
 - 4.6 Stacking Ensemble
 - 4.7 Decision Threshold Optimisation
 - 4.8 Probability Calibration
- 5. Model Evaluation
- 6. Inference Pipeline
- 7. API Reference
- 8. Web Interface
- 9. Deployment
- 10. Project Structure
- 11. Reproducibility
- 12. Limitations & Future Work
- 13. References

1. Project Overview

This project develops a machine-learning system that estimates an individual's **10-year risk of coronary heart disease (CHD)** from routine clinical measurements. The goal is to provide a calibrated, interpretable probability rather than a binary verdict — enabling clinicians to weigh risk in context rather than act on a hard threshold alone.

Key design principles:

- **Leakage-free pipeline** — imputation statistics, scalers, and feature-selection decisions are fitted exclusively on training data and serialised for reuse at inference time.
- **Clinical threshold tuning** — the decision threshold is selected to maximise F2-score (recall-weighted) on a held-out validation set, reflecting the asymmetric cost of false negatives in cardiac screening.
- **Probability calibration** — Platt scaling corrects for overconfident or underconfident raw probabilities, so that a model output of 0.30 reflects a $\approx 30\%$ real-world incidence.
- **Production-ready serving** — a FastAPI backend serves the trained model bundle; a single-page HTML/JS interface communicates with the API with no build step required.

2. Dataset

Property	Detail
Source	Framingham Heart Study (public version)
File	data_cardiovascular_risk.csv
Cohort size	$\sim 3,390$ patients
Follow-up horizon	10 years
Target variable	TenYearCHD (binary: 1 = CHD event, 0 = no event)
Class balance	Approximately 15 % positive (heavily imbalanced)

2.1 FEATURE INVENTORY

Feature	Type	Description
age	Continuous	Patient age in years (20–90)
sex	Categorical	Biological sex (M / F)
education	Ordinal	1 = Some HS · 2 = HS/GED · 3 = Some College · 4 = Degree
is_smoking	Binary	Current active smoker
cigsPerDay	Continuous	Average cigarettes per day
BPMeds	Binary	Currently on antihypertensive medication
prevalentStroke	Binary	Prior cerebrovascular event
prevalentHyp	Binary	Pre-existing hypertension diagnosis

Feature	Type	Description
diabetes	Binary	Confirmed diabetes mellitus (Type I or II)
totChol	Continuous	Total cholesterol (mg/dL)
sysBP	Continuous	Systolic blood pressure (mmHg)
diaBP	Continuous	Diastolic blood pressure (mmHg)
BMI	Continuous	Body mass index (kg/m²)
heartRate	Continuous	Resting heart rate (bpm)
glucose	Continuous	Fasting blood glucose (mg/dL)

2.2 MISSING VALUES

Column	Strategy
glucose, totChol, cigsPerDay, BMI, heartRate	Median imputation (computed from training fold only)
education, BPMeds	Mode imputation (computed from training fold only)

3. System Architecture

[illegible]

4. Methodology

4.1 EXPLORATORY DATA ANALYSIS

The EDA phase (run automatically via `train.py`) produces:

- **Class distribution plot** — visualises the label imbalance (~85 % / 15 %).
- **Missing value heatmap** — identifies columns with non-trivial missingness.
- **Continuous distributions** — histograms + KDE for all eight continuous features, stratified by CHD outcome.
- **Correlation heatmap** — Pearson correlations on raw numeric features to flag potential multicollinearity before feature selection.

4.2 DATA PREPROCESSING PIPELINE

All preprocessing strictly follows a **no-leakage** protocol: every statistic (median, mode, IQR fences, variance, correlation) is computed on the training fold and then applied to the validation and test folds.

```
Raw DataFrame ■ ■ ■ Stratified 3-way split ■ 60 % train · 20 % validation · 20 % test ■ ■ ■
Median / mode imputation (fit on X_train) ■ ■ ■ One-hot encoding (deterministic: sex,
is_smoking) ■ ■ ■ Phase 1 feature engineering (deterministic, §4.3) ■ ■ ■ Phase 2 feature
engineering (fit on X_train: IQR clipping, column selection) ■ ■ ■ Standard scaling (fit on
X_train, zero-mean unit-variance) ■ ■ ■ SMOTE oversampling (training fold only, §4.4)
```

Split rationale:

The three-way split separates threshold tuning (validation set) from final evaluation (test set). This prevents the threshold — itself a hyperparameter — from inflating test-set metrics.

4.3 FEATURE ENGINEERING

Feature engineering is divided into two phases to preserve the leakage-free property.

Phase 1 — Deterministic transforms (applied identically to all splits):

Engineered Feature	Formula / Logic	Clinical Rationale
pulsePressure	sysBP – diaBP	Validated marker of arterial stiffness; replaces raw systolic/diastolic
age_group	Binned into <40, 40–50, 50–60, ≥60 (ACC/AHA bands)	Non-linear age risk is better captured by categorical bands
is_obese	BMI ≥ 30 → 1	WHO obesity threshold as binary flag
is_overweight	25 ≤ BMI < 30 → 1	WHO overweight threshold
smoking_intensity	is_smoking × cigsPerDay	Interaction term capturing pack-year exposure
Log-transform	log1p applied to cigsPerDay, totChol, BMI, heartRate, glucose, pulsePressure	Reduces right-skew; clip(lower=0) guards against negatives

Phase 2 — Statistical transforms (fit on training data only):

	Method	Threshold
Normalisation)	IQR fences $Q1 - 1.5 \times IQR$ / $Q3 + 1.5 \times IQR$	Per-column, computed on X_{train}
Column removal	Variance filter	$var < 0.01$
Row removal	Pairwise Pearson upper triangle	$r > 0.90$

4.4 CLASS IMBALANCE HANDLING

The dataset carries an approximate **15 % positive rate**, which causes most classifiers to be biased toward the majority class. SMOTE (Synthetic Minority Over-sampling Technique) is applied exclusively to the training fold after scaling:

SMOTE generates synthetic minority-class samples by interpolating between existing positive-class neighbours in feature space. Applied after scaling ensures the synthetic points are in the same normalised space as real data.

The validation and test sets are **never resampled** — they retain the original class distribution to give realistic evaluation metrics.

4.5 MODEL TRAINING & HYPERPARAMETER TUNING

Eight base classifiers are trained and tuned via **stratified 5-fold GridSearchCV** on the SMOTE-resampled training set, scoring on **ROC-AUC**.

Model	Key Hyperparameter Grid
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$, $l1_ratio \in \{0, 1\}$ (ElasticNet via SAGA)
Random Forest	$n_estimators \in \{100, 200, 300\}$, $max_depth \in \{5, 10, 15, None\}$
XGBoost	$max_depth \in \{3, 5, 7\}$, $learning_rate \in \{0.01, 0.1, 0.2\}$, $n_estimators \in \{100, 200, 300\}$
K-Nearest Neighbours	$k \in \{3, 5, 7, 11\}$, $weights \in \{uniform, distance\}$, $metric \in \{euclidean, manhattan\}$
Support Vector Classifier	$C \in \{0.1, 1, 10\}$, $kernel \in \{rbf, linear\}$
Gaussian Naïve Bayes	$var_smoothing \in \{1e-9 \dots 1e-5\}$
Gradient Boosting	$n_estimators \in \{100, 200\}$, $learning_rate \in \{0.05, 0.1, 0.2\}$, $max_depth \in \{3, 5\}$

Model	Key Hyperparameter Grid
AdaBoost	$n_estimators \in \{50, 100, 200\}$, $learning_rate \in \{0.5, 1.0, 1.5\}$

Scoring metric: ROC-AUC was chosen for tuning because it is threshold-independent and robust to class imbalance.

4.6 STACKING ENSEMBLE

After individual tuning, a **stacking classifier** is constructed from the three highest-AUC base models:

```
Base layer: Top-3 models by ROC-AUC (fitted estimators, passthrough=False)
Meta-learner: Logistic Regression trained on out-of-fold predictions
Training data: train ∪ val (combined, 5-fold CV used internally by sklearn)
```

The stacker is evaluated on the same held-out test set as all base models.

4.7 DECISION THRESHOLD OPTIMISATION

Raw probabilistic classifiers default to a 0.50 cut-off, which is sub-optimal for imbalanced clinical data where **missing a true positive is far more costly than a false alarm**.

The optimal threshold is selected by scanning $[0.10, 0.90]$ in steps of 0.01 and maximising the **F2-score** ($\beta = 2$, which weights recall twice as heavily as precision) on the **validation set**, subject to a minimum precision floor of 0.20:

```
For each threshold t: y_pred = (y_prob ≥ t)
if precision(y_pred) < 0.20: skip ← prevent degenerate "flag everyone" solution
score = F2(y_true, y_pred)
t* = argmax(score)
```

The minimum precision floor prevents the pathological solution of labelling every patient positive (which maximises recall at zero precision).

4.8 PROBABILITY CALIBRATION

Tree-based models and SVMs are often poorly calibrated — their raw output probabilities do not reflect actual event frequencies. **Platt scaling** is applied to the best model using the validation set as the calibration set:

```
Step 1: Generate raw probability scores from the fitted model on X_val.
Step 2: Fit a logistic regression f: raw_prob → calibrated_prob using (raw_probs, y_val).
Step 3: Wrap the base model in _PlattCalibratedModel, which chains both.
```

The base model weights are **never modified**. Only the probability mapping is adjusted. After calibration, the threshold is re-optimised on the calibrated probability scores.

Benefit: A calibrated model output of 0.25 should reflect a $\approx 25\%$ empirical event rate — critical for communicating risk to clinicians rather than raw scores.

5. Model Evaluation

METRICS COMPUTED

Metric	Formula	Relevance
ROC-AUC	Area under ROC curve	Threshold-independent discriminative ability
Sensitivity (Recall)	$TP / (TP + FN)$	Fraction of true cases detected — primary concern
Specificity	$TN / (TN + FP)$	Fraction of true negatives correctly dismissed
Precision	$TP / (TP + FP)$	Positive predictive value
F1-Score	Harmonic mean of precision and recall	Balanced metric
F2-Score	Recall-weighted F-measure ($\beta=2$)	Tuning objective; emphasises recall

FINAL MODEL PERFORMANCE (LOGISTIC REGRESSION, CALIBRATED)

Metric	Value
ROC-AUC	0.714
Sensitivity	72.6 %
Specificity	58.0 %
Decision Threshold	0.18
Cohort	3,390 patients

EVALUATION PLOTS GENERATED

- Confusion matrix (per model)
- ROC curves (all models overlaid)
- Calibration curves (reliability diagrams) — before and after Platt scaling
- Learning curve (training vs. validation AUC over dataset size)

- Feature importance bar chart (tree-based models)
 - Model comparison table and bar chart
 - Decision-curve analysis (net benefit vs. treat-all / treat-none baselines)
 - 5-fold cross-validation summary
-

6. Inference Pipeline

At inference time, the serialised `best_model_bundle.joblib` contains all fitted objects needed to reproduce the exact training transforms:

```
bundle = { "model": <PlattCalibratedModel>, "fill_values": {col: median/mode}, #  
imputation "stat_params": {fences, selector}, # IQR clipper + feature selector "scaler":  
<StandardScaler>, "feature_cols": [list of column names], # alignment at inference  
"model_name": str, "threshold": float, }
```

Inference steps (`preprocess_inference`):

1. Apply median/mode imputation using stored `fill_values`
 2. One-hot encode `sex` and `is_smoking` (`drop_first=False` to preserve all columns on single-row inputs)
 3. Apply deterministic Phase-1 feature engineering (identical to training)
 4. Winsorise using stored `stat_params["fences"]`
 5. Align columns to `feature_cols`, filling any absent columns with 0
 6. Scale using stored `StandardScaler`
 7. Call `model.predict_proba(X)[: , 1]` → apply threshold → return label + probability
-

7. API Reference

The FastAPI server (`app_api.py`) exposes four endpoints.

GET /

Serves the web UI (`Cardiovascular_Risk_Assessment.html`).

GET /HEALTH

Returns the loaded model name and server status.

Response:

```
{ "status": "ok", "model": "LogisticRegression" }
```

POST /PREDICT

Single-patient prediction.

Request body:

```
{ "age": 55, "sex": 1, "is_smoking": 0, "cigsPerDay": 0, "BPMeds": 0, "prevalentStroke": 0,
  "prevalentHyp": 1, "diabetes": 0, "totChol": 250, "sysBP": 140, "diaBP": 90, "BMI": 28.5,
  "heartRate": 75, "glucose": 100, "education": 2 }
```

Response:

```
{ "model_name": "LogisticRegression", "threshold": 0.18, "chd_probability": 0.3142,
  "chd_prediction": 1, "risk_label": "High Risk" }
```

POST /PREDICT/BATCH

Accepts a JSON array of patient objects. Returns per-patient index, probability, prediction, and label.

Input field constraints:

Field	Type	Range
age	float	20 – 100
sex	int	0 (F) / 1 (M)
is_smoking	int	0 / 1
cigsPerDay	float	0 – 100
BPMeds	int	0 / 1
prevalentStroke	int	0 / 1
prevalentHyp	int	0 / 1
diabetes	int	0 / 1
totChol	float	100 – 700
sysBP	float	80 – 300
diaBP	float	40 – 200
BMI	float	10 – 60
heartRate	float	30 – 200

Field	Type	Range
glucose	float	40 – 500
education	float	1 – 4

Interactive Swagger docs: <http://localhost:8000/docs>

8. Web Interface

The single-page UI (`Cardiovascular_Risk_Assessment.html`) is served directly by the FastAPI app at `GET /`. No build step, no framework — pure HTML, CSS, and vanilla JavaScript.

Features:

- **Age slider** — large typographic display updates in real time.
- **Segmented toggle** — sex selection.
- **Bistate switches** — five binary condition indicators with animated toggles.
- **Conditional field** — cigarettes/day row expands only when "Current Smoker" is active.
- **API health check** — on page load, the UI pings `GET /health` and shows "API CONNECTED" or falls back to "DEMO MODE" with a client-side logistic approximation.
- **Gauge canvas** — semicircular D3-style gauge drawn with the Canvas API, colour-coded Low / Moderate / High.
- **Risk factor breakdown table** — all 11+ input features graded Normal / Elevated / High with clinical reference ranges.
- **Clinical recommendations** — rule-based recommendations generated client-side from the risk factor breakdown and probability outcome.
- **Offline demo mode** — if the API is unreachable, a JavaScript logistic approximation runs in-browser so the UI is always demonstrable.

9. Deployment

LOCAL DEVELOPMENT

```
# Install dependencies pip install -r requirements.txt # Start server uvicorn app_api:app
--host 0.0.0.0 --port 8000 --reload # Open browser http://localhost:8000
```

PRODUCTION — RENDER (FREE TIER)

The repository includes `render.yaml` for zero-configuration deployment:

```
services: - type: web name: cardiovascular-risk-predictor env: python plan: free
buildCommand: pip install -r requirements.txt startCommand: uvicorn app_api:app --host
0.0.0.0 --port $PORT healthCheckPath: /health
```

Steps:

1. Push repository to GitHub.
2. Log into render.com → New → Web Service → connect repo.
3. Render detects `render.yaml` automatically → click **Deploy**.

Free-tier behaviour: The service sleeps after 15 minutes of inactivity. The first request after sleep triggers a cold start (~30 s). Subsequent requests are fast.

10. Project Structure

```
■■■■ app_api.py # FastAPI app – serving UI + prediction endpoints ■■■■ app_streamlit.py #
Alternative Streamlit UI (local use) ■■■■ Cardiovascular Risk Assessment.html # Production
web UI ■■■■ render.yaml # Render deployment configuration ■■■■ requirements.txt # API
runtime dependencies ■■■■ models/ ■■■■ best_model_bundle.joblib # Serialised model + all
preprocessing artifacts ■■■■ src/ ■■■■ config.py # Paths, hyperparameter grids, column
lists ■■■■ data_loader.py # CSV loading, basic EDA printing ■■■■ preprocessing.py # Full
train/inference preprocessing pipelines ■■■■ feature_engineering.py # Phase 1
(deterministic) + Phase 2 (statistical) ■■■■ models.py # Model registry, GridSearchCV,
stacking, calibration ■■■■ evaluation.py # Metrics, cross-validation, decision-curve
analysis ■■■■ visualization.py # All plots (confusion matrix, ROC, calibration, etc.) ■
■■■■ train.py # End-to-end training entry point ■■■■ predict.py # Standalone CLI predictor
(uses bundle) ■■■■ utils.py # Seed, timer, CSV saver ■■■■ reports/ # Auto-generated CSVs
and plots from train.py
```

11. Reproducibility

All random operations are seeded via a single constant (`RANDOM_STATE = 42`) passed to every scikit-learn estimator, SMOTE, and train/test split. The model bundle serialises all fitted preprocessing objects alongside the model, guaranteeing that inference results are identical regardless of the Python environment, as long as library versions satisfy `requirements.txt`.

To retrain from scratch:

```
# Requires data_cardiovascular_risk.csv in the project root python src/train.py
```

This overwrites `models/best_model_bundle.joblib` and regenerates all plots and CSVs in `reports/`.

12. Limitations & Future Work

Limitation	Detail
Dataset scope	Framingham study cohort is predominantly White American; generalisability to other ethnicities is unestablished.
Static model	The bundle is a snapshot; there is no continuous retraining or model drift detection.
Feature scope	LDL/HDL ratio, family history, physical activity, and medication adherence are absent from the dataset but clinically relevant.
Binary outcome	The model predicts any CHD event; it does not distinguish MI severity, onset timing, or mortality risk.
Cold-start latency	Free-tier Render deployment sleeps after inactivity — not suitable for time-critical clinical tooling.

Potential extensions:

- Swap the static threshold for an **expected-value threshold** that weights false-negative cost against false-positive cost explicitly.
- Add **SHAP explainability** to the API response so each prediction is accompanied by per-feature contributions.
- Implement **model monitoring** with statistical drift detection on incoming feature distributions.
- Integrate **GP or Bayesian calibration** as an alternative to Platt scaling for small calibration sets.
- Package with Docker for environment-independent deployment.

13. References

1. Dawber, T. R., Meadors, G. F., & Moore, F. E. (1951). Epidemiological approaches to heart disease: the Framingham Study. *American Journal of Public Health*, 41(3), 279–286.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3), 61–74.
4. Vuk, M., & Curk, T. (2006). ROC curve, lift chart and calibration plot. *Metodološki zvezki*, 3(1), 89–108.

5. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*, 785–794.

This instrument is intended for research and educational purposes. Its outputs do not constitute medical advice, diagnosis, or treatment, and must not replace evaluation by a qualified clinician.

Author: Akshay Bawaliwale

Repository: github.com/AkshayAI007/Cardiovascular-disease-risk-prediction-using-Machine-learning

Year: 2026

This instrument is intended for research and educational purposes only. Its outputs do not constitute medical advice, diagnosis, or treatment.